S10 · Observabilidad y diagnóstico

Jannys Graciela Navarro Acrota

Equipo de referencia: 09 · Casa o Nada. Tarea personal sobre Pagatu.

Aporte documentado: evolución de órdenes, cliente y pago, seguridad, comunicación, verificación y documentación individual con asistencia de herramientas.

Ejecución real: 5 de noviembre de 2026 · America/Lima. Fechas ISO en UTC pueden figurar como 6 de noviembre.

Repositorio GitHub · Stephmarc/Tareas-Distribuidas

Evidencia técnica · cuatro bloques

1. Health groups reales

Cliente configura livenessState en liveness y readinessState,db en readiness. Antes de la falla ambos devuelven 200 UP. Se detuvo el contenedor PostgreSQL de cliente, manteniendo vivo su proceso Java.

2. Logs y correlación

El mismo X-Trace-ID de una orden aparece en Loki en catálogo, órdenes, pago y cliente. El filtro lo almacena en MDC; Feign lo propaga por HTTP y los eventos lo llevan como traceId. El JSON original incluye las cuatro etiquetas de servicio y sus líneas.

3. Tablero propio

Grafana usa Prometheus y Loki provisionados desde obs. Incluye disponibilidad, tasa de peticiones/errores y logs por servicio. La captura es del tablero con series y logs reales, no de la pantalla vacía de bienvenida.

4. Diagnóstico de falla

Con PostgreSQL detenido, liveness respondió 200 UP, readiness 503 DOWN y GET clientes 500. Loki registró Connection refused hacia localhost:15434 y agotamiento de conexiones Hikari. Al recuperar el mismo contenedor, readiness volvió a 200 UP sin borrar datos.

Tabla de diagnóstico: antes 200/200; falla de PostgreSQL 200/503 y negocio 500; recuperación 200/200. El panel de disponibilidad mide scraping, no capacidad completa de negocio.

Código real del repositorio

pagatu-cliente-ms-dev.yml

Infra/pagatu-config/config-repo/pagatu-cliente-ms-dev.yml

```
server:
  port: 8083
spring:
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:15434/pagatu_cliente_db}
    username: ${DB_USER:pagatu}
    password: ${DB_PASSWORD:pagatu}
  jpa:
    hibernate:
      ddl-auto: validate
```

pagatu-cliente-ms-dev.yml

Infra/pagatu-config/config-repo/pagatu-cliente-ms-dev.yml

```
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      probes:
        enabled: true
      group:
        liveness:
          include: livenessState
        readiness:
          include: readinessState,db
logging:
  file:
    name: logs/cliente.log
  pattern:
    level: "%5p [traceId=%X{traceId}]"

spring.kafka.bootstrap-servers: ${KAFKA_BOOTSTRAP:localhost:19092}
spring.kafka.consumer.group-id: pagatu-cliente
spring.kafka.consumer.auto-offset-reset: earliest
spring.kafka.consumer.key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
```

CorrelationIdFilter.java

Services/pagatu-cliente-ms/src/main/java/pe/edu/upeu/cliente/CorrelationIdFilter.java

```
    HttpServletResponse r,
    FilterChain c
  ) throws ServletException, IOException {
    String id = q.getHeader("X-Trace-ID");
    if (id == null || !id.matches("[A-Za-z0-9._-]{1,100}")) id =
      UUID.randomUUID().toString();
    MDC.put("traceId", id);
    r.setHeader("X-Trace-ID", id);
    try {
      LoggerFactory.getLogger(getClass()).info(
        "{} {}",
        q.getMethod(),
        q.getRequestURI()
      );
      c.doFilter(q, r);
    } finally {
      MDC.remove("traceId");
    }
  }
}
```

Ejecución y resultados originales

Comando reproducible: & .\scripts\Run-PersonalEvidence.ps1 -Sesion s10

Los transcripts son salida real de PowerShell y los JSON provienen de las respuestas de la aplicación. Se adjuntan completos en MkDocs. No se sustituyen por un texto de aprobación.

Cliente liveness antes de falla | {"status": 200, "response": {"status": "UP"}}

Cliente readiness antes de falla | {"status": 200, "response": {"status": "UP"}}

Proceso vivo con PostgreSQL detenido | {"status": 200, "response": {"status": "UP"}}

Readiness detecta dependencia PostgreSQL real caida | {"status": 503, "response": {"status": "DOWN"}}

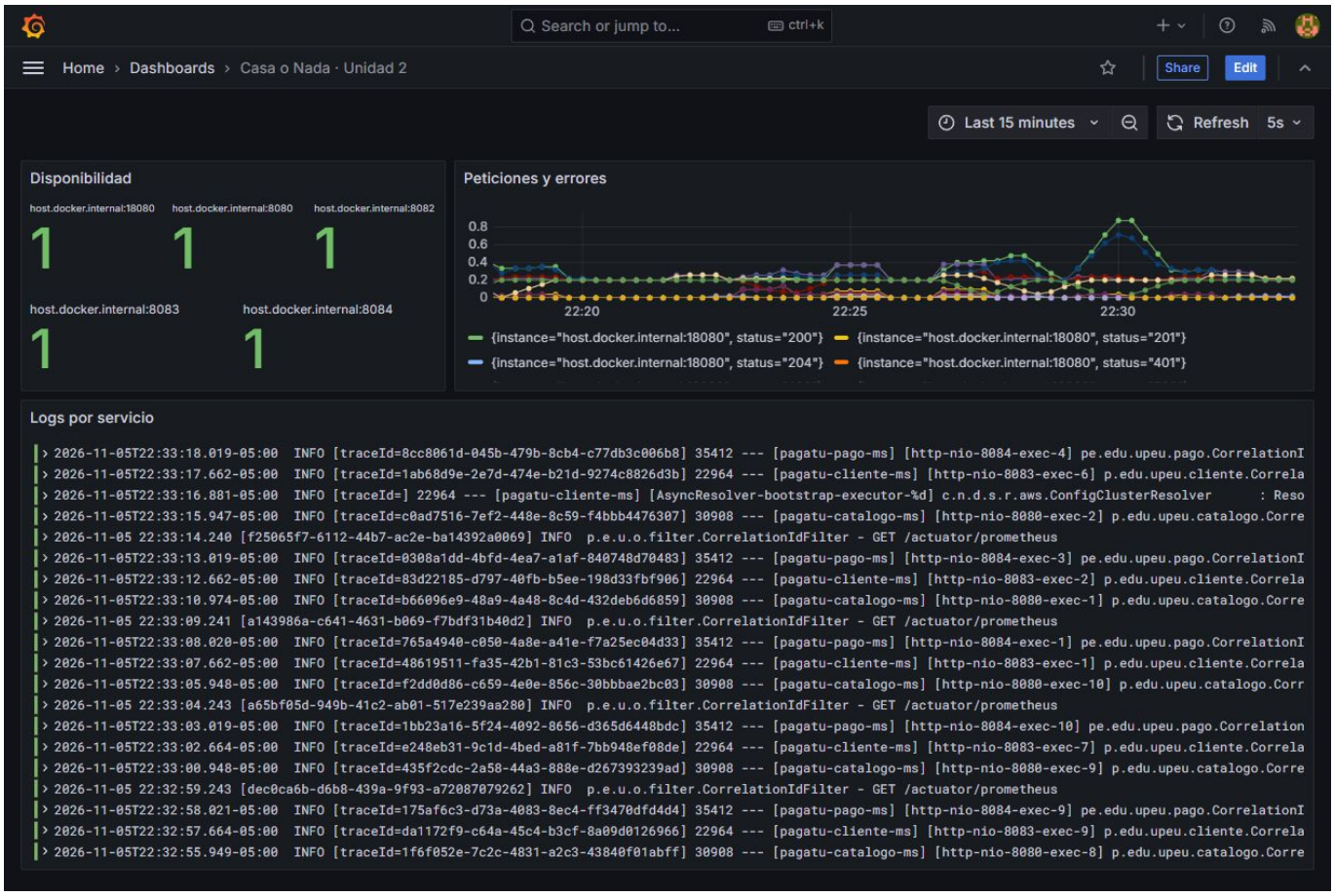
Peticion dependiente de BD falla de forma observable | {"status": 500, "response": {"timestamp": "2026-11-06T03:32:35.043Z", "status": 500, "error": "Internal Server Error", "path": "/api/v1/clientes"}}

Readiness vuelve a UP tras recuperar PostgreSQL | {"status": 200, "response": {"status": "UP"}}

Prometheus recoge objetivos reales | {"data": [{"metric": {"__name__": "up", "instance": "host.docker.internal:8083", "job": "pagatu"}, "value": [1793935960.693, "1"]}, {"metric": {"__name__": "up", "instance": "host.docker.internal:8084", "job": "pagatu"}, "value": [1793935960.693, "1"]}, {"metric": {"__name__": "up", "instance": "host.docker.internal:19000", "job": "keycloak"}, "value": [1793935960.693, "1"]}, {"metric": {"__name__": "up", "instance": "host.docker.internal:8082", "job": "pagatu"}, "value": [1793935960.693, "1"]}, {"metric": {"__name__": "up", "instance": "host.docker.internal:18080", "job": "pagatu"}, "value": [1793935960.693, "1"]}, {"metric": {"__name__": "up", "instance": "host.docker.internal:8080", "job": "pagatu"}, "value": [1793935960.693, "1"]}]}

Consulta real a Loki con trace de orden | {"status": 200, "streams": [{"application": "pagatu-catalogo-ms", "detected_level": "info", "filename": "/logs/catalogo/catalogo.log", "service_name": "pagatu-catalogo-ms"}, {"application": "pagatu-cliente-ms", "detected_level": "info", "filename": "/logs/cliente/cliente.log", "service_name": "pagatu-cliente-ms"}, {"application": "pagatu-orden-ms", "detected_level": "info", "filename": "/logs/orden/orden.log", "service_name": "pagatu-orden-ms"}, {"application": "pagatu-pago-ms", "detected_level": "info", "filename": "/logs/pago/pago.log", "service_name": "pagatu-pago-ms"}]}

Captura real



Captura íntegra, sin edición de resultados ni fechas. Grafana después de recuperar PostgreSQL: disponibilidad del proceso, series de códigos 500/503 durante la ventana de falla y logs centralizados. No se presenta como una captura tomada mientras la base seguía detenida.. La salida original completa está vinculada en MkDocs.

Hallazgo y reflexión técnica

Error o hallazgo real

Un objetivo Prometheus up=1 solo indica que pudo recolectarse su endpoint. En la caída real de PostgreSQL el proceso siguió disponible pero el negocio respondió 500. Por eso se comparan readiness, códigos HTTP y logs, no solo disponibilidad del proceso.

Reflexión técnica

Un proceso puede contestar health aunque su base esté detenida.

Liveness comprueba que sigue vivo; readiness comprueba si puede atender.

La prueba separó ambas condiciones con PostgreSQL realmente apagado.

Los errores 500 y los logs de conexión explicaron el fallo de negocio.

La correlación permitió unir la petición con los servicios participantes.

Knight Capital recuerda que ejecutar código no garantiza operar correctamente.

Recuperar readiness confirma la dependencia, pero también se debe revisar el resultado de negocio.

Límite de alcance: esta entrega personal usa Pagatu; la adaptación al Proyecto Sello grupal CASAS se evalúa por separado. No se atribuye una defensa oral realizada ni una nota de evaluación.

Anexo · Feedback de la sesión

Campos personales pendientes de completar por Jannys. No se inventan respuestas sobre la experiencia, comprensión, esfuerzo o satisfacción en clase.

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

3. ¿Tienes alguna pregunta para la siguiente clase?

4. Nivel de comprensión: Entendido / Más o menos / Necesito ayuda.

5. ¿Cómo puede ayudarte el docente a comprender mejor el tema?

6. Participación: Muy comprometido/a / Comprometido/a / Poco comprometido/a.

7. Satisfacción con la clase: 1 a 10.